

13a — Docker (& LXC)

Key Concepts

- **Docker** = Containerization platform — packages app, code, libraries, config into a container
 - Solves "works on my machine" — identical environment everywhere
 - **LXC** = Low-level Linux technology Docker is built on (namespaces + cgroups) — nobody uses it directly anymore
-

Core Building Blocks

Concept	Definition
Dockerfile	Text file with build instructions — the recipe
Image	Read-only snapshot built from Dockerfile — the template
Container	Running instance of an image — the live process
Registry	Storage and distribution system for images

```
Dockerfile → docker build → Image → docker run → Container  
                                     (live, running process)
```

One image can spawn many containers simultaneously.

Dockerfile Example (.NET)

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0  
WORKDIR /app  
COPY . .  
RUN dotnet build  
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

- Each instruction adds a **layer** to the image
 - Layers are **cached** — rebuilds only redo changed layers (fast)
-

Common Docker Commands

```
docker build -t myapp .           # build image from Dockerfile  
docker run -p 8080:80 myapp       # run container, map ports  
docker push myapp:latest          # push image to registry
```

```
docker pull myapp:latest      # pull image from registry
docker ps                    # list running containers
docker stop <id>              # stop a container
```

Docker Compose

Tool for running **multiple containers together** via a single YAML config file:

```
services:
  api:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
  cache:
    image: redis:7
```

```
docker compose up    # starts all services together
docker compose down  # stops and removes all containers
```

Use case: Local development — mirrors production architecture locally.

VM vs Container

	Virtual Machine	Container
Isolation	Full OS isolation	Process-level isolation
Size	GBs	MBs
Boot time	Minutes	Seconds
OS	Each has its own full OS	Shares host OS kernel
Security	Stronger isolation	Slightly weaker
Use case	Different OSes, strong isolation	Microservices, fast deploys

Docker Registries

Registry	Used by
----------	---------

Registry	Used by
Docker Hub	Public images, open source
AWS ECR	Amazon ECS/EKS workloads
Azure Container Registry	Azure workloads
GitHub Container Registry	GitHub Actions pipelines

LXC vs Docker

	LXC	Docker
Level	Low-level Linux primitive	High-level developer tool
Ease of use	Complex, manual config	Simple CLI and Dockerfile
Use case	System containers (lightweight VMs)	App containers

Industry Examples

Company	Docker Usage
Netflix	All microservices in containers — thousands deployed daily
Uber	Each microservice is a Docker container managed at scale
Spotify	Migrated from VMs to Docker — deployment time dropped from hours to minutes
GitHub Actions	Every CI job runs in a Docker container — clean, isolated, reproducible

Interview Questions

Q1: What problem does Docker solve?

Environment inconsistency — "works on my machine." Packages app + dependencies into a container that runs identically everywhere.

Q2: VM vs Container — key difference?

VM runs a full OS per instance — GBs, minutes to boot. Container shares the host OS kernel — MBs, seconds to boot.

Q3: Dockerfile vs Image vs Container?

Dockerfile = build instructions. Image = read-only snapshot built from Dockerfile. Container = running instance of an image.

Q4: What is a Docker Registry?

Storage and distribution system for images. Docker Hub is public; AWS ECR and Azure Container Registry are common in production.

Q5: What is Docker Compose?

Tool for running multi-container apps via a single YAML config. One `docker compose up` starts the whole stack — used to mirror production locally.

Q6: What is LXC and how does it relate to Docker?

LXC is the low-level Linux technology Docker is built on — uses kernel namespaces and cgroups for isolation. Docker abstracts it into a developer-friendly platform.